

Parallel and Distributed Computing Assignment Report

Overview

This project focuses on implementing a particle simulation in a two-dimensional space, with the objective of exploring different computational approaches for solving the n -body problem. The simulation models the movement of particles under the influence of gravitational forces, using a Particle-in-Cell (PIC) approximation to reduce computational complexity.

The project consists of three main implementations:

- A serial implementation, serving as the baseline for performance comparison.
- A parallel implementation using OpenMP, optimizing computation for shared-memory architectures.
- A distributed-memory implementation using MPI, enabling scalability across multiple compute nodes.

The goal is to analyze and compare the efficiency of these implementations by measuring execution time across different problem sizes and parallel configurations. The project also involves addressing key challenges such as load balancing, synchronization, and communication overhead.

The implementation is done in C, utilizing OpenMP and MPI for parallelism. The report will include a detailed discussion on the methodologies employed, performance evaluations, and insights gained from the parallelization strategies.

Part 1 - Serial Implementation

Initially, different serial code versions were implemented:

1. Using a vector of indexes of particles for each cell.
2. Using a vector of particles for each cell.
3. Using vectors of properties of particles for each cell.

The third version turned out to be the best one in terms of speedup performance, due to better spatial locality and cache exploitation; therefore, it is the only one described in this report.

The serial version of the simulation follows an approach where the space is divided into a grid, and iterates over each time step sequentially computing the following cells' properties:

- **Computing Centers of Mass:** each grid cell computes the weighted center of mass from the particles it contains.
- **Calculating Kinetics:** the forces acting on each particle are determined based on interactions with particles within the same cell and adjacent grid cells. For this purpose, an extra border was created to manage the interactions between the centers of mass and the particles, as the space around the grid would be contiguous. Using the resultant force on each particle, the position and velocities are updated.
- **Computing new Particle Cell:** after updating the particle's position, it may move to a different cell. In such cases, the particle must be transferred to the data structure of the new cell.
- **Handling Collisions:** if two particles come within a threshold distance or have been pushed outside of the cell, they are marked as collided and removed.

Furthermore, to explore more spatial parallelism, the original particles vector was transformed into a vector of `cell_t`, where `cell_t` is a custom struct. This struct contains multiple vectors - one for each property of the particles within a given cell. As a result, all for-loops in the four steps of the algorithm now iterate over the cells, rather than individual particles, significantly improving spatial locality.

Additionally, this specialized data structure enables the use of SIMD instructions to operate on vectors of four *long long* elements, further increasing performance. Initially, `#pragma omp simd` was used to enable SIMD execution, but HPC Analysis with VTune revealed that no SIMD instructions were actually being utilized. Consequently, a new version of the code was developed explicitly using SIMD instructions via AVX-2 Intrinsics.

Part 2 - OpenMP Implementation

To improve performance, the serial implementation was improved using OpenMP directives. The following parallelization strategies were applied:

- **Creating one Parallel Region:** `#pragma parallel` directive is applied only once outside the main for loop that handles each time step of the simulation. Thus, minimizing the overhead with thread creation.
- **Loop Parallelization:** `#pragma omp for` directive is applied to distribute iterations of loops among threads, where each iteration processes a different cell.
- **Work Scheduling:** since each cell can have a different number of particles, a dynamic scheduling strategy is used to balance workloads across threads. However, the static and guided modes are also tested, in some cases static shows a performance slightly better than dynamic. After multiple benchmarks, the optimal chunk size found was 2 or 1 depending on the number of rows of our grid.
- **Data Race Avoidance:** an OpenMP lock is utilized to synchronize access to shared memory regions, preventing race conditions. In this case, a lock for read and write operations of the cells vectors. To reduce the overhead, each cell has its own mutex, so when a cell wants to add a particle in another cell, it needs to get the target cell mutex.
- **Reduction for Accumulators:** The collision count is accumulated using OpenMP's reduction clause to ensure correct summation across multiple threads.

Results

The Table 1 and 2 shows the time and speed up of each test case. The speedup is computed between the serial version and the OpenMP version using different thread numbers each time.

Test Case	Serial	OMP (1 Thr.)	OMP (2 Thr.)	OMP (4 Thr.)	OMP (8 Thr.)
[12 100 5 10000 10000]	29.9s	29.7s	15.3s	8.1s	5.9s
[-11 3500 20 500000 10]	24.8s	24.8s	12.4s	6.2s	4.6s
[1 5000 100 1000000 4]	0.9s	0.9s	0.5s	0.3s	0.3s
[1 5000 100 1000000 100]	19.6s	19.6s	10.0s	5.3s	3.6s
[1 5000 20 1000000 10]	28.1s	28.1s	14.1s	7.1s	4.9s
[1 1000 3 10000 10000]	128.3s	127.8s	70.0s	41.6s	29.5s
[3 5000 50 1000000 300]	160.6s	155.1s	78.2s	39.5s	26.6s
[3 5000 50 1000000 500]	260.3s	258.4s	130.0s	65.7s	44.3s
[-1 1000 30 100000 1000]	44.4s	43.8s	22.1s	11.2s	7.4s

Table 1: Execution Time (in seconds) for Serial and OpenMP Versions

Test Case	1 Thread	2 Threads	4 Threads	8 Threads
[12 100 5 10000 10000]	1.00	1.95	3.69	5.06
[-11 3500 20 500000 10]	1.00	2.00	4.00	5.39
[1 5000 100 1000000 4]	1.00	1.80	3.00	3.00
[1 5000 100 1000000 100]	1.00	1.96	3.69	5.44
[1 5000 20 1000000 10]	1.00	1.99	3.95	5.73
[1 1000 3 10000 10000]	1.00	1.83	3.08	4.34
[3 5000 50 1000000 300]	1.03	2.05	4.06	6.03
[3 5000 50 1000000 500]	1.00	2.00	3.96	5.87
[-1 1000 30 100000 1000]	1.01	2.00	3.96	6.00

Table 2: Speedup Summary for Different Test Cases

The best performance is obtained in every test using 8 threads, as shown in Figure 1.

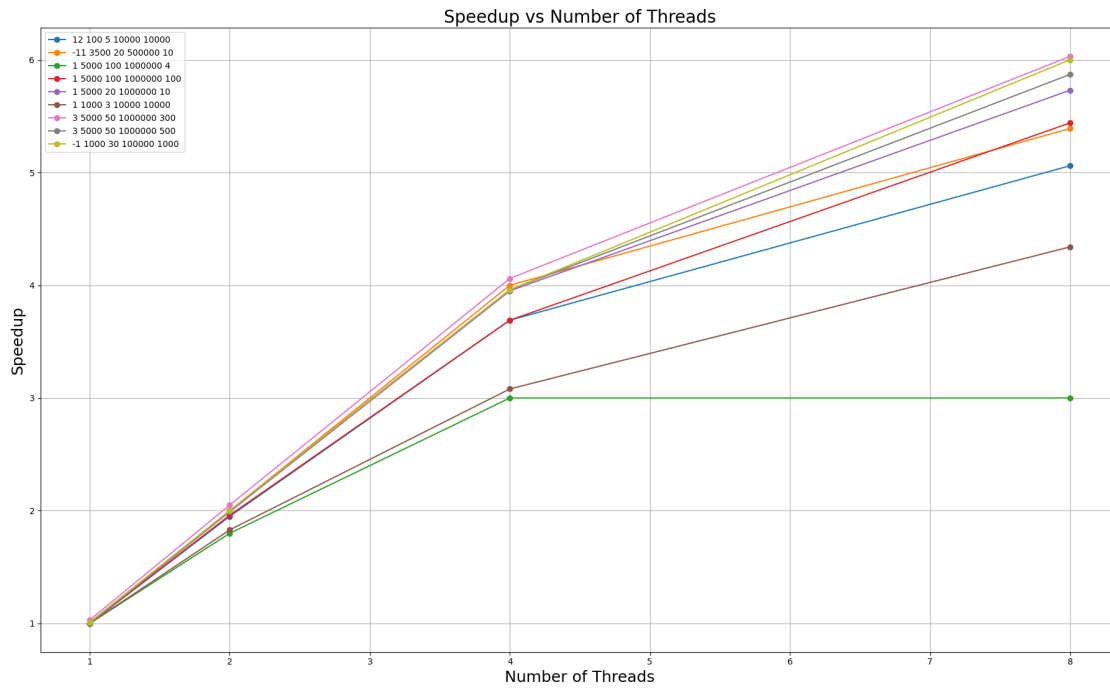


Figure 1: Speedup related to the Number of Thread

In general, a positive speedup is reached thanks to these key factors:

- **Thread-level parallelism:** independent workloads are distributed across multiple CPU cores, reducing execution time.
- **Dynamic scheduling:** ensures better load balance by dynamically assigning iterations to available threads.
- **Parallel reduction:** efficiently accumulates results across threads.
- **Minimized synchronization overhead:** Each cell has its own lock, thus reducing the competition by resources.

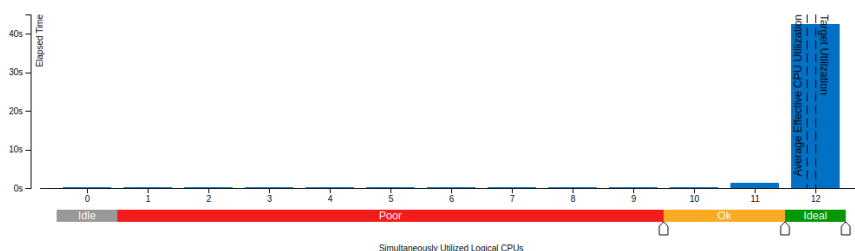
However, from 4 to 8 threads, the speed up did not scale linearly. This is partially because the serial version is highly optimized, reducing its execution time. As a result, when work is divided among threads, each workload may become very small, especially for lower particle counts. Additionally, some test cases involve a large number of time steps, requiring frequent synchronization barriers, which further reduce performance. In particular, the case [1,1000,3,10000,10000] exhibits minimal speedup between 4 and 8 threads due to the combination of many time steps and a low number of particles.

Using V-Tune Intel tool with Threading functionality was verified that a high CPU (98.9%) Utilization was achieved with low overhead (0.5%), see Figure 2 showing the analysis for the test with the longer execution time, which is [3 5000 50 1000000 500].

Effective CPU Utilization: 98.9% (11.863 out of 12 logical CPUs)

Effective CPU Utilization Histogram

This histogram displays a percentage of the wall time the specific number of CPUs were running simultaneously. Spin and Overhead time adds to the Idle CPU utilization value.



Total Thread Count: 12

Thread Oversubscription: 0s (0.0% of CPU Time)

Wait Time with poor CPU Utilization: 0.000s (100.0% of Wait Time)

Top Waiting Objects

This section lists the objects that spent the most time waiting in your application. Objects can wait on specific calls, such as sleep() or I/O, or on contented synchronizations. A significant amount of Wait time associated with a synchronization object reflects high contention for that object and, thus, reduced parallelism.

Sync Object	Wait Time with poor CPU Utilization	(% from Object Wait Time)	Wait Count
Stream 0xad5e57f4	0.000s	100.0%	1

*NA is applied to non-summable metrics.

Spin and Overhead Time: 2.660s (0.5% of CPU Time)

Figure 2: Vtune Threading Analysis for test [3 5000 50 1000000 500]

Conclusion

This project explored the implementation of a particle simulation using both a serial approach and an OpenMP parallel implementation. The results demonstrated an improvement in execution time when the implementation was shifted from the serial to the OpenMP version, as expected. The speedup analysis in Table 2 and Figure 1 shows that increasing the number of threads leads to substantial performance improvement. However, the actual speedup varies depending on the problem size and the number of particles chosen.

Note that the ideal speed up of 8 (considering the 8 thread parallelization) is never reached. One possible reason is that the overhead introduced by parallelization (e.g. thread management, synchronization, and memory contention) reduces the computational benefits, especially since the serial version has already been highly optimized.